Session 26 — TEAMWORK ANSWER KEY Walkthrough

This code-first guide follows the authoritative classroom implementation sequence exactly. Apply each step in order; file paths, code, commands, and identifiers are reproduced unchanged.

TW-01 — `createSprintAction` -- the real validation skeleton

Step: `TW-01`

GOAL

A real new export exists that reads a real sprint-creation form and validates it, before it ever calls the real backend.

TECHNICAL CONTEXT

The identical real Server-Action-form shape LIVE just taught on `createIssueAction` -- transferring it to a second new real resource in the same session is the point.

FILE

`web/src/lib/actions/sprints.ts`

EXACT LOCATION

A brand-new file, in the existing `lib/actions/` folder.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```tsx
"use server";

import { requireSessionToken } from "@/lib/session";

export interface SprintFormState {
  error?: string;
}

export async function createSprintAction(
```

```
  projectId: number,
  basePath: string,
  _prevState: SprintFormState,
  formData: FormData,
): Promise<SprintFormState> {
  const token = await requireSessionToken();
  const name = String(formData.get("name") ?? "").trim();
  const goal = String(formData.get("goal") ?? "").trim();

  if (!name) {
    return { error: "Name is required." };
  }

  // The real backend call comes next (TW-02).
  return {};
}
```

AFTER

TSX

```tsx
"use server";

import { requireSessionToken } from "@/lib/session";

export interface SprintFormState {
  error?: string;
}

export async function createSprintAction(
  projectId: number,
  basePath: string,
  _prevState: SprintFormState,
  formData: FormData,
): Promise<SprintFormState> {
  const token = await requireSessionToken();
  const name = String(formData.get("name") ?? "").trim();
  const goal = String(formData.get("goal") ?? "").trim();

  if (!name) {
    return { error: "Name is required." };
  }

  // The real backend call comes next (TW-02).
  return {};
}
```

WHY THIS WORKS

`goal` has no real `required` -- the real backend's own `CreateSprintDto` already treats it as optional; the form matches that real contract exactly.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing calls the real backend yet (TW-02).

TW-02 — `createSprintAction` -- the real backend call

Step: `TW-02`

GOAL

A real new sprint is actually created against the real backend.

TECHNICAL CONTEXT

Pure, mechanical continuation of TW-01 -- the real validation already exists; this step only calls the real `POST /projects/:projectId/sprints` route.

FILE

`web/src/lib/actions/sprints.ts`

EXACT LOCATION

The `import` list grows by one; the final `return {};` placeholder is replaced by the real backend call.

BEFORE

TSX

```
import { requireSessionToken } from "@/lib/session";
```

TSX

```
  // The real backend call comes next (TW-02).
  return {};
}
```

IMPLEMENTATION

TSX

```tsx
import { revalidatePath } from "next/cache";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
```

TSX

```tsx
  const res = await apiFetch(`/projects/${projectId}/sprints`, {
    method: "POST",
    token,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, goal }),
  });

  if (!res.ok) {
    return { error: "Something went wrong. Please try again." };
  }

  revalidatePath(`${basePath}/sprints`);
  return {};
}
```

AFTER

TSX

```tsx
import { revalidatePath } from "next/cache";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
```

TSX

```tsx
  const res = await apiFetch(`/projects/${projectId}/sprints`, {
    method: "POST",
    token,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, goal }),
  });

  if (!res.ok) {
    return { error: "Something went wrong. Please try again." };
  }

  revalidatePath(`${basePath}/sprints`);
  return {};
}
```

WHY THIS WORKS

`revalidatePath` targets `${basePath}/sprints`, never the generic issues list -- the real Sprints page is the only real view this action's own result needs to invalidate.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing calls this yet (TW-05).

TW-03 — `startSprintAction`

Step: `TW-03`

GOAL

A real new export exists that starts a real planned sprint -- not yet called from anywhere.

TECHNICAL CONTEXT

Unlike `createSprintAction`, nothing here is ever submitted from a real `<form>` -- a real "Start sprint" button (TW-14) calls this directly with values it already has in scope, the identical real shape `updateMemberRoleAction` / `removeMemberAction` already used in Session 25.

FILE

`web/src/lib/actions/sprints.ts`

EXACT LOCATION

A brand-new export, added at the end of the file, after `createSprintAction`.

BEFORE

TSX

```
  revalidatePath(`${basePath}/sprints`);
  return {};
}
```

IMPLEMENTATION

TSX

```
  revalidatePath(`${basePath}/sprints`);
  return {};
}

export async function startSprintAction(sprintId: number, basePath: string) {
  const token = await requireSessionToken();

  const res = await apiFetch(`/sprints/${sprintId}/start`, {
    method: "POST",
    token,
  });

  if (!res.ok) {
    const body: { message?: string } = await res.json().catch(() => ({}));
    throw new Error(body.message ?? "Could not start the sprint.");
  }

  revalidatePath(`${basePath}/sprints`);
}
```

AFTER

TSX

```
  revalidatePath(`${basePath}/sprints`);
  return {};
}

export async function startSprintAction(sprintId: number, basePath: string) {
  const token = await requireSessionToken();

  const res = await apiFetch(`/sprints/${sprintId}/start`, {
    method: "POST",
    token,
  });

  if (!res.ok) {
    const body: { message?: string } = await res.json().catch(() => ({}));
    throw new Error(body.message ?? "Could not start the sprint.");
  }

  revalidatePath(`${basePath}/sprints`);
}
```

WHY THIS WORKS

The real error message is read from the real backend's own JSON body (`body.message`), not a generic string -- the real one-active-sprint `409` the backend throws names the real sprint that is already active, which is worth showing verbatim.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing calls this yet (TW-14).

TW-04 — `completeSprintAction`

Step: `TW-04`

GOAL

A real new export exists that completes a real active sprint -- not yet called from anywhere.

TECHNICAL CONTEXT

Pure, mechanical continuation of TW-03 -- the identical real shape, for the real other half of the sprint lifecycle.

FILE

`web/src/lib/actions/sprints.ts`

EXACT LOCATION

A brand-new export, added at the end of the file, after `startSprintAction` .

BEFORE

TSX

```
  revalidatePath(`${basePath}/sprints`);
}
```

IMPLEMENTATION

TSX

```
  revalidatePath(`${basePath}/sprints`);
}

export async function completeSprintAction(sprintId: number, basePath: string) {
  const token = await requireSessionToken();

  const res = await apiFetch(`/sprints/${sprintId}/complete`, {
    method: "POST",
    token,
  });

  if (!res.ok) {
    const body: { message?: string } = await res.json().catch(() => ({}));
    throw new Error(body.message ?? "Could not complete the sprint.");
  }

  revalidatePath(`${basePath}/sprints`);
}
```

AFTER

TSX

```
  revalidatePath(`${basePath}/sprints`);
}

export async function completeSprintAction(sprintId: number, basePath: string) {
  const token = await requireSessionToken();

  const res = await apiFetch(`/sprints/${sprintId}/complete`, {
    method: "POST",
    token,
  });

  if (!res.ok) {
    const body: { message?: string } = await res.json().catch(() => ({}));
    throw new Error(body.message ?? "Could not complete the sprint.");
  }

  revalidatePath(`${basePath}/sprints`);
}
```

WHY THIS WORKS

Forward-only: this action only ever moves a sprint `active -> completed` -- the real backend itself rejects completing anything that is not currently `active` (e.g. a real double-complete, or completing a still-`planned` sprint), so the frontend never needs its own duplicate check.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing calls this yet (TW-16).

TW-05 — `CreateSprintForm` -- the real action wiring

Step: `TW-05`

GOAL

A real form shell exists, wired to `createSprintAction` through `useActionState` -- no real fields yet.

TECHNICAL CONTEXT

The identical real `useActionState` shape `CreateIssueForm` just used -- proving the pattern transfers to a third real resource in the same session.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/CreateSprintForm.tsx`

EXACT LOCATION

A brand-new file, in a new `sprints/` folder.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```
"use client";
```

```
import { useActionState } from "react";
import { createSprintAction, type SprintFormState } from "@/lib/actions/sprints";

const initialState: SprintFormState = {};

export function CreateSprintForm({
  projectId,
  basePath,
}: {
  projectId: number;
  basePath: string;
}) {
  const action = createSprintAction.bind(null, projectId, basePath);
  const [state, formAction, pending] = useActionState(action, initialState);

  return <form action={formAction} className="flex flex-col gap-2" />;
}
```

AFTER

TSX

```
"use client";

import { useActionState } from "react";
import { createSprintAction, type SprintFormState } from "@/lib/actions/sprints";

const initialState: SprintFormState = {};

export function CreateSprintForm({
  projectId,
  basePath,
}: {
  projectId: number;
  basePath: string;
}) {
  const action = createSprintAction.bind(null, projectId, basePath);
  const [state, formAction, pending] = useActionState(action, initialState);

  return <form action={formAction} className="flex flex-col gap-2" />;
}
```

WHY THIS WORKS

`createSprintAction.bind(null, projectId, basePath)` -- the identical real way `CreateIssueForm` binds extra real arguments ahead of the two `useActionState` itself always supplies.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real Name field, error display, and submit button come next (TW-06).

TW-06 — `CreateSprintForm` -- the real name field and submit

Step: `TW-06`

GOAL

The real form now collects a real name, shows a real backend error, and offers a real way to submit.

TECHNICAL CONTEXT

Pure, mechanical continuation of TW-05 -- the identical real field/error/submit shape `CreateIssueForm` already used.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/CreateSprintForm.tsx`

EXACT LOCATION

The self-closing `<form />` grows real children: an error paragraph, the Name field, and a submit button.

BEFORE

TSX

```
  return <form action={formAction} className="flex flex-col gap-2" />;
```

IMPLEMENTATION

TSX

```
  return (
    <form action={formAction} className="flex flex-col gap-2">
      {state.error && (
        <p className="text-sm text-red-600 dark:text-red-400">{state.error}</p>
```

```
      )}
      <label className="flex flex-col gap-1 text-sm">
        Name
        <input
          name="name"
          type="text"
          required
          placeholder="Sprint 1"
          className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
        />
      </label>
      <button
        type="submit"
        disabled={pending}
        className="rounded-md border border-blue-600 bg-blue-600 px-3 py-1 text-sm font-
medium text-white disabled:opacity-50"
      >
        {pending ? "Creating..." : "Create sprint"}
      </button>
    </form>
  );
```

AFTER

TSX

```
  return (
    <form action={formAction} className="flex flex-col gap-2">
      {state.error && (
        <p className="text-sm text-red-600 dark:text-red-400">{state.error}</p>
      )}
      <label className="flex flex-col gap-1 text-sm">
        Name
        <input
          name="name"
          type="text"
          required
          placeholder="Sprint 1"
          className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
        />
      </label>
      <button
        type="submit"
        disabled={pending}
        className="rounded-md border border-blue-600 bg-blue-600 px-3 py-1 text-sm font-
```

```
medium text-white disabled:opacity-50"
      >
        {pending ? "Creating..." : "Create sprint"}
      </button>
    </form>
  );
```

WHY THIS WORKS

`placeholder="Sprint 1"` -- a real, concrete example value, since "Sprint" alone (unlike a workspace or issue title) genuinely benefits from showing the expected real naming convention.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. No real goal field yet (TW-07); nothing renders this component yet (TW-12).

TW-07 — `CreateSprintForm` -- the real goal field

Step: `TW-07`

GOAL

The real form also collects an optional goal.

TECHNICAL CONTEXT

Pure, mechanical continuation of TW-06 -- one more real field, the identical real shape.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/CreateSprintForm.tsx`

EXACT LOCATION

Right after the real Name `<label>`, before the submit `<button>`.

BEFORE

TSX

```
      </label>
      <button
        type="submit"
```

IMPLEMENTATION

TSX

```
      </label>
      <label className="flex flex-col gap-1 text-sm">
        Goal (optional)
        <input
          name="goal"
          type="text"
          className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
        />
      </label>
      <button
        type="submit"
```

AFTER

TSX

```
      </label>
      <label className="flex flex-col gap-1 text-sm">
        Goal (optional)
        <input
          name="goal"
          type="text"
          className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
        />
      </label>
      <button
        type="submit"
```

WHY THIS WORKS

A plain `<input>`, not a `<textarea>` -- unlike an issue's own `description`, a real sprint goal is meant to stay a short, one-line real statement.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing renders this component yet (TW-12).

TW-08 — Sprints page -- the real skeleton

Step: `TW-08`

GOAL

A real, visitable page exists that fetches a project's own real data -- no real sprints yet.

TECHNICAL CONTEXT

The identical real page shape every other project-scoped page already uses: fetch the real project, guard with `notFound()`.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/page.tsx`

EXACT LOCATION

A brand-new file at `.../sprints/page.tsx`, with no real sprints yet.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import type { Project } from "@/lib/types";

export default async function SprintsPage({
  params,
}: {
  params: Promise<{ id: string; projectId: string }>;
```

```
}) {
  const { projectId } = await params;

  const projectRes = await apiFetch(`/projects/${projectId}`);
  if (!projectRes.ok) {
    notFound();
  }
  const project: Project = await projectRes.json();

  return <main>{project.key}</main>;
}
```

AFTER

TSX

```tsx
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import type { Project } from "@/lib/types";

export default async function SprintsPage({
  params,
}: {
  params: Promise<{ id: string; projectId: string }>;
}) {
  const { projectId } = await params;

  const projectRes = await apiFetch(`/projects/${projectId}`);
  if (!projectRes.ok) {
    notFound();
  }
  const project: Project = await projectRes.json();

  return <main>{project.key}</main>;
}
```

WHY THIS WORKS

The real page re-fetches the real project itself, rather than receiving it as a prop -- the identical real convention every Server Component page in this app already follows.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real sprints fetch comes next (TW-09).

TW-09 — Sprints page -- the real sprints fetch

Step: `TW-09`

GOAL

The real page also fetches the project's own real sprints -- not yet rendered.

TECHNICAL CONTEXT

Pure, mechanical continuation of TW-08 -- the identical real auth-required fetch shape every other related-list page already uses.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/page.tsx`

EXACT LOCATION

Two real new imports; the real `params` / `project` already resolved grow a real `basePath` and a real sprints fetch.

BEFORE

TSX

```tsx
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import type { Project } from "@/lib/types";
```

TSX

```tsx
  const { projectId } = await params;
```

TSX

```tsx
  const project: Project = await projectRes.json();

  return <main>{project.key}</main>;
}
```

IMPLEMENTATION

TSX

```tsx
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
import type { Project, Sprint } from "@/lib/types";
```

TSX

```tsx
  const { id, projectId } = await params;
  const token = await requireSessionToken();
```

TSX

```tsx
  const project: Project = await projectRes.json();
  const basePath = `/workspaces/${id}/projects/${project.id}`;

  const sprintsRes = await apiFetch(`/projects/${project.id}/sprints`, { token });
  const sprints: Sprint[] = await sprintsRes.json();

  return <main>{project.key}</main>;
}
```

AFTER

TSX

```tsx
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
import type { Project, Sprint } from "@/lib/types";
```

TSX

```tsx
  const { id, projectId } = await params;
  const token = await requireSessionToken();
```

TSX

```tsx
  const project: Project = await projectRes.json();
  const basePath = `/workspaces/${id}/projects/${project.id}`;

  const sprintsRes = await apiFetch(`/projects/${project.id}/sprints`, { token });
  const sprints: Sprint[] = await sprintsRes.json();

  return <main>{project.key}</main>;
}
```

WHY THIS WORKS

`{ token }` is passed explicitly on the real sprints fetch -- the real `GET /projects/:id/sprints` route genuinely requires auth (any real workspace member), unlike several earlier S23/S24 pages that read real public data.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing renders the real back link or heading yet (TW-10); the real list comes after (TW-11).

TW-10 — Sprints page -- the real back link and heading

Step: `TW-10`

GOAL

The real page now renders a real back link and a real heading -- no real list yet.

TECHNICAL CONTEXT

Pure, mechanical continuation of TW-09 -- the identical real header shape every other project-scoped page already uses.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/page.tsx`

EXACT LOCATION

One real new import; the placeholder `<main>` grows real content.

BEFORE

TSX

```
import { notFound } from "next/navigation";
```

TSX

```
  return <main>{project.key}</main>;
}
```

IMPLEMENTATION

TSX

```
import Link from "next/link";
import { notFound } from "next/navigation";
```

TSX

```
  return (
    <main className="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <Link href={basePath} className="text-sm text-blue-600 hover:underline dark:text-blue-
400">
        Back to project
      </Link>
      <h1 className="text-xl font-semibold">{project.key} — Sprints</h1>
    </main>
  );
}
```

AFTER

TSX

```
import Link from "next/link";
import { notFound } from "next/navigation";
```

TSX

```
  return (
    <main className="mx-auto flex max-w-2xl flex-col gap-4 p-6">
      <Link href={basePath} className="text-sm text-blue-600 hover:underline dark:text-blue-
400">
        Back to project
      </Link>
      <h1 className="text-xl font-semibold">{project.key} — Sprints</h1>
    </main>
  );
}
```

WHY THIS WORKS

The real heading reads `{project.key} — Sprints`, the identical real convention the real Board page's own heading already uses.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real sprint list comes next (TW-11).

TW-11 — Sprints page -- the real list and points summary

Step: `TW-11`

GOAL

The real page now lists every real sprint, each with its own real points summary.

TECHNICAL CONTEXT

Pure, mechanical continuation of TW-10 -- the real data already exists; this step only renders it.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/page.tsx`

EXACT LOCATION

One real new import; the real `<h1>` grows a real sibling `<div>` that lists every real sprint.

BEFORE

TSX

```
import type { Project, Sprint } from "@/lib/types";
```

TSX

```
      <h1 className="text-xl font-semibold">{project.key} -- Sprints</h1>
    </main>
```

IMPLEMENTATION

TSX

```
import type { Project, Sprint } from "@/lib/types";
import { Card } from "@/components/ui/Card";
```

TSX

```
      <h1 className="text-xl font-semibold">{project.key} — Sprints</h1>

      <div className="flex flex-col gap-3">
        {sprints.map((sprint) => (
          <Card key={sprint.id}>
            <p className="font-medium">{sprint.name}</p>
            <p className="text-sm text-gray-500 dark:text-gray-400">
              {sprint.goal || "No goal set."}
            </p>
            <p className="mt-1 text-xs text-gray-400">
              {sprint.status} — {sprint.points.completedPoints}/{sprint.points.totalPoints}
points done
            </p>
          </Card>
        ))}
        {sprints.length === 0 && (
          <p className="text-sm text-gray-400">No sprints yet.</p>
        )}
      </div>
    </main>
```

AFTER

TSX

```
import type { Project, Sprint } from "@/lib/types";
import { Card } from "@/components/ui/Card";
```

TSX

```
      <h1 className="text-xl font-semibold">{project.key} — Sprints</h1>

      <div className="flex flex-col gap-3">
        {sprints.map((sprint) => (
          <Card key={sprint.id}>
            <p className="font-medium">{sprint.name}</p>
            <p className="text-sm text-gray-500 dark:text-gray-400">
              {sprint.goal || "No goal set."}
            </p>
            <p className="mt-1 text-xs text-gray-400">
```

```
              {sprint.status} -- {sprint.points.completedPoints}/{sprint.points.totalPoints}
points done
            </p>
          </Card>
        ))}
        {sprints.length === 0 && (
          <p className="text-sm text-gray-400">No sprints yet.</p>
        )}
      </div>
    </main>
```

WHY THIS WORKS

`sprint.points` is read straight off the real backend's own nested shape -- never re-derived client-side from a real issue list the page does not even fetch.

COMMANDS

BASH

```
npx tsc --noEmit && npm run build
```

VERIFY

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1/sprints -o
/dev/null -w "%{http_code}\n"
```

EXPECTED RESULT

`tsc` / `build` both exit clean; real `200`. No real create form yet (TW-12).

TW-12 — Sprints page -- the real create-sprint card

Step: `TW-12`

GOAL

The real Sprints page also offers the real create-sprint form.

TECHNICAL CONTEXT

Pure wiring -- `CreateSprintForm` already exists (TW-05..07); this step renders it.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/page.tsx`

EXACT LOCATION

One real new import; a new `<Card>` right after the closing `</div>` of the real sprint list.

BEFORE

TSX

```
import { Card } from "@/components/ui/Card";
```

TSX

```
        )}
      </div>
    </main>
  );
}
```

IMPLEMENTATION

TSX

```
import { Card } from "@/components/ui/Card";
import { CreateSprintForm } from "./CreateSprintForm";
```

TSX

```
        )}
      </div>

      <Card>
        <h2 className="mb-3 text-sm font-semibold uppercase text-gray-500 dark:text-gray-
400">
          Create a sprint
        </h2>
        <CreateSprintForm projectId={project.id} basePath={basePath} />
      </Card>
    </main>
  );
}
```

AFTER

TSX

```
import { Card } from "@/components/ui/Card";
import { CreateSprintForm } from "./CreateSprintForm";
```

TSX

```
        )}
      </div>

      <Card>
        <h2 className="mb-3 text-sm font-semibold uppercase text-gray-500 dark:text-gray-
400">
          Create a sprint
        </h2>
        <CreateSprintForm projectId={project.id} basePath={basePath} />
      </Card>
    </main>
  );
}
```

WHY THIS WORKS

The create-sprint card is its own `Card`, a sibling of the sprint list -- not nested inside it, the identical real visual grouping Workspace Settings (Session 25) already established.

COMMANDS

BASH

```
npx tsc --noEmit && npm run build
```

VERIFY

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1/sprints |
grep -o "Create a sprint"
```

EXPECTED RESULT

`tsc` / `build` both exit clean; the real create-sprint form renders.

TW-13 — `SprintLifecycleControls` -- the real props and state

Step: `TW-13`

GOAL

A real component exists, typed for one real sprint's own lifecycle controls, with no real button yet.

TECHNICAL CONTEXT

What new problem are you solving? A sprint's own real status decides which ONE real action makes sense ("planned" -> Start; "active" -> Complete; "completed" -> neither) -- the forward- only lifecycle the real backend itself enforces (`SprintsService`). This component mirrors that real decision in the UI, never contradicts it, even though the real enforcement stays server-side.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/SprintLifecycleControls.tsx`

EXACT LOCATION

A brand-new file at `.../sprints/SprintLifecycleControls.tsx` , with no real button yet.

BEFORE

File does not exist yet.

IMPLEMENTATION

TSX

```tsx
"use client";

import { useState } from "react";

// Forward-only: "planned" only ever offers Start; "active" only ever offers Complete;
// "completed" offers neither -- the real backend's own forward-only lifecycle guard (Sprint can
// never go active -> planned or completed -> active) is mirrored here, never contradicted, even
// though the real enforcement lives server-side (SprintsService), not in this component.
export function SprintLifecycleControls({
  sprintId,
  status,
  basePath,
}: {
  sprintId: number;
  status: string;
  basePath: string;
}) {
  const [pending, setPending] = useState(false);
```

```
  const [error, setError] = useState<string | undefined>();

  return (
    <div className="flex flex-col gap-1">
      {error && <p className="text-xs text-red-600 dark:text-red-400">{error}</p>}
    </div>
  );
}
```

AFTER

TSX

```
"use client";

import { useState } from "react";

// Forward-only: "planned" only ever offers Start; "active" only ever offers Complete;
// "completed" offers neither -- the real backend's own forward-only lifecycle guard (Sprint can
// never go active -> planned or completed -> active) is mirrored here, never contradicted, even
// though the real enforcement lives server-side (SprintsService), not in this component.
export function SprintLifecycleControls({
  sprintId,
  status,
  basePath,
}: {
  sprintId: number;
  status: string;
  basePath: string;
}) {
  const [pending, setPending] = useState(false);
  const [error, setError] = useState<string | undefined>();

  return (
    <div className="flex flex-col gap-1">
      {error && <p className="text-xs text-red-600 dark:text-red-400">{error}</p>}
    </div>
  );
}
```

WHY THIS WORKS

One real shared `error` / `pending` pair, not one per real button -- only one real lifecycle action is ever even offered at a time (by real status), so there is never a real case where two real attempts could be in flight together.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. No real button yet (TW-14..17).

TW-14 — `SprintLifecycleControls` -- the real `start()` handler

Step: `TW-14`

GOAL

A real `start()` handler exists that calls `startSprintAction` -- not yet offered through any real button.

TECHNICAL CONTEXT

Pure, mechanical continuation of TW-13 -- the real shared state already exists; this step adds the real handler that will drive the real Start button.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/SprintLifecycleControls.tsx`

EXACT LOCATION

One real new import; a real `start()` function added right before the `return`.

BEFORE

TSX

```
import { useState } from "react";
```

TSX

```
  return (
    <div className="flex flex-col gap-1">
      {error && <p className="text-xs text-red-600 dark:text-red-400">{error}</p>}
```

```
    </div>
  );
```

IMPLEMENTATION

TSX

```tsx
import { useState } from "react";
import { startSprintAction } from "@/lib/actions/sprints";
```

TSX

```tsx
  async function start() {
    setPending(true);
    setError(undefined);
    try {
      await startSprintAction(sprintId, basePath);
    } catch (e) {
      setError(e instanceof Error ? e.message : "Could not start the sprint.");
    } finally {
      setPending(false);
    }
  }

  return (
    <div className="flex flex-col gap-1">
      {error && <p className="text-xs text-red-600 dark:text-red-400">{error}</p>}
    </div>
  );
```

AFTER

TSX

```tsx
import { useState } from "react";
import { startSprintAction } from "@/lib/actions/sprints";
```

TSX

```tsx
  async function start() {
    setPending(true);
    setError(undefined);
    try {
      await startSprintAction(sprintId, basePath);
    } catch (e) {
      setError(e instanceof Error ? e.message : "Could not start the sprint.");
    } finally {
      setPending(false);
    }
```

```
  }

  return (
    <div className="flex flex-col gap-1">
      {error && <p className="text-xs text-red-600 dark:text-red-400">{error}</p>}
    </div>
  );
```

WHY THIS WORKS

The real error is set from `startSprintAction`'s own thrown `Error` message -- the real one-active-sprint `409` the backend returns names the real sprint that is already active, real information worth showing, not a generic fallback.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real Start button comes next (TW-15).

TW-15 — `SprintLifecycleControls` -- the real Start button

Step: `TW-15`

GOAL

A real "planned" sprint offers a real, working Start button.

TECHNICAL CONTEXT

Pure, mechanical continuation of TW-14 -- the real handler already exists; this step only adds the real conditional button that calls it.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/SprintLifecycleControls.tsx`

EXACT LOCATION

One real conditional button added inside the returned `<div>`.

BEFORE

TSX

```
    <div className="flex flex-col gap-1">
      {error && <p className="text-xs text-red-600 dark:text-red-400">{error}</p>}
    </div>
  );
```

IMPLEMENTATION

TSX

```
    <div className="flex flex-col gap-1">
      {error && <p className="text-xs text-red-600 dark:text-red-400">{error}</p>}
      {status === "planned" && (
        <button
          type="button"
          disabled={pending}
          onClick={start}
          className="rounded-md border border-blue-600 px-2 py-1 text-xs font-medium text-
blue-600 disabled:opacity-50 dark:text-blue-400"
        >
          Start sprint
        </button>
      )}
    </div>
  );
```

AFTER

TSX

```
    <div className="flex flex-col gap-1">
      {error && <p className="text-xs text-red-600 dark:text-red-400">{error}</p>}
      {status === "planned" && (
        <button
          type="button"
          disabled={pending}
          onClick={start}
          className="rounded-md border border-blue-600 px-2 py-1 text-xs font-medium text-
blue-600 disabled:opacity-50 dark:text-blue-400"
        >
          Start sprint
        </button>
      )}
```

```
    </div>
  );
```

WHY THIS WORKS

`disabled={pending}` -- a real double-click must not fire a second real `startSprintAction` call while the first is still in flight.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real Complete handler comes next (TW-16); nothing renders this component yet (TW-18).

TW-16 — `SprintLifecycleControls` -- the real `complete()` handler

Step: `TW-16`

GOAL

A real `complete()` handler exists that calls `completeSprintAction` -- not yet offered through any real button.

TECHNICAL CONTEXT

Pure, mechanical continuation of TW-15 -- the identical real shape as `start()`, for the real other half of the lifecycle.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/SprintLifecycleControls.tsx`

EXACT LOCATION

One real new import; a real `complete()` function added after `start()`.

BEFORE

TSX

```
import { startSprintAction } from "@/lib/actions/sprints";
```

TSX

```
  }

  return (
```

IMPLEMENTATION

TSX

```
import { startSprintAction, completeSprintAction } from "@/lib/actions/sprints";
```

TSX

```
  }

  async function complete() {
    setPending(true);
    setError(undefined);
    try {
      await completeSprintAction(sprintId, basePath);
    } catch (e) {
      setError(e instanceof Error ? e.message : "Could not complete the sprint.");
    } finally {
      setPending(false);
    }
  }

  return (
```

AFTER

TSX

```
import { startSprintAction, completeSprintAction } from "@/lib/actions/sprints";
```

TSX

```
  }

  async function complete() {
    setPending(true);
    setError(undefined);
    try {
      await completeSprintAction(sprintId, basePath);
    } catch (e) {
      setError(e instanceof Error ? e.message : "Could not complete the sprint.");
    } finally {
      setPending(false);
    }
```

```
  }

  return (
```

WHY THIS WORKS

`complete()` is the identical real shape as `start()`, calling `completeSprintAction` instead -- real evidence the pattern needed no new real idea, only a second real call site.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. The real Complete button comes next (TW-17).

TW-17 — `SprintLifecycleControls` -- the real Complete button

Step: `TW-17`

GOAL

A real "active" sprint offers a real, working Complete button.

TECHNICAL CONTEXT

Pure, mechanical continuation of TW-16 -- the identical real shape as the Start button, for the real other half of the lifecycle.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/SprintLifecycleControls.tsx`

EXACT LOCATION

One real conditional button added right after the "planned" button, inside the same `<div>`.

BEFORE

TSX

```
          Start sprint
        </button>
```

```
      )}
    </div>
  );
```

IMPLEMENTATION

TSX

```
          Start sprint
        </button>
      )}
      {status === "active" && (
        <button
          type="button"
          disabled={pending}
          onClick={complete}
          className="rounded-md border border-green-600 px-2 py-1 text-xs font-medium text-
green-600 disabled:opacity-50 dark:text-green-400"
        >
          Complete sprint
        </button>
      )}
    </div>
  );
```

AFTER

TSX

```
          Start sprint
        </button>
      )}
      {status === "active" && (
        <button
          type="button"
          disabled={pending}
          onClick={complete}
          className="rounded-md border border-green-600 px-2 py-1 text-xs font-medium text-
green-600 disabled:opacity-50 dark:text-green-400"
        >
          Complete sprint
        </button>
      )}
    </div>
  );
```

WHY THIS WORKS

A real second, independent condition (`status === "active"`) -- never an `else` branch off the "planned" check, since a real "completed" sprint must show neither button.

VERIFY

BASH

```
npx tsc --noEmit
```

EXPECTED RESULT

Clean exit. Nothing renders this component yet (TW-18).

TW-18 — wire `SprintLifecycleControls` into the Sprints page

Step: `TW-18`

GOAL

Every real sprint row now shows its own real lifecycle control.

TECHNICAL CONTEXT

Pure wiring -- the real component already exists (TW-13..17); this step renders it per sprint.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/sprints/page.tsx`

EXACT LOCATION

One real new import; the sprint `<Card>` 's own returned JSX grows a real `<div>` wrapper and the real controls.

BEFORE

TSX

```
import { CreateSprintForm } from "./CreateSprintForm";
```

TSX

```
          <Card key={sprint.id}>
            <p className="font-medium">{sprint.name}</p>
            <p className="text-sm text-gray-500 dark:text-gray-400">
              {sprint.goal || "No goal set."}
```

```
            </p>
            <p className="mt-1 text-xs text-gray-400">
              {sprint.status} -- {sprint.points.completedPoints}/{sprint.points.totalPoints}
points done
            </p>
          </Card>
```

IMPLEMENTATION

TSX

```
import { CreateSprintForm } from "./CreateSprintForm";
import { SprintLifecycleControls } from "./SprintLifecycleControls";
```

TSX

```
          <Card key={sprint.id}>
            <div className="flex items-center justify-between gap-3">
              <div>
                <p className="font-medium">{sprint.name}</p>
                <p className="text-sm text-gray-500 dark:text-gray-400">
                  {sprint.goal || "No goal set."}
                </p>
                <p className="mt-1 text-xs text-gray-400">
                  {sprint.status} --
{sprint.points.completedPoints}/{sprint.points.totalPoints} points done
                </p>
              </div>
              <SprintLifecycleControls
                sprintId={sprint.id}
                status={sprint.status}
                basePath={basePath}
              />
            </div>
          </Card>
```

AFTER

TSX

```
import { CreateSprintForm } from "./CreateSprintForm";
import { SprintLifecycleControls } from "./SprintLifecycleControls";
```

TSX

```
          <Card key={sprint.id}>
            <div className="flex items-center justify-between gap-3">
              <div>
                <p className="font-medium">{sprint.name}</p>
```

```
                <p className="text-sm text-gray-500 dark:text-gray-400">
                  {sprint.goal || "No goal set."}
                </p>
                <p className="mt-1 text-xs text-gray-400">
                  {sprint.status} --
{sprint.points.completedPoints}/{sprint.points.totalPoints} points done
                </p>
              </div>
              <SprintLifecycleControls
                sprintId={sprint.id}
                status={sprint.status}
                basePath={basePath}
              />
            </div>
          </Card>
```

WHY THIS WORKS

The text block is wrapped in its own real `<div>`, a sibling of `SprintLifecycleControls` -- needed so `justify-between` can separate the real text from the real controls, the identical real layout shape `MemberRow` (Session 25) already used.

COMMANDS

BASH

```
npx tsc --noEmit && npm run build
```

VERIFY

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1/sprints |
grep -o "Start sprint"
```

EXPECTED RESULT

`tsc` / `build` both exit clean; a real "planned" sprint shows its own real Start button.

TW-19 — link to the real Sprints page; TEAMWORK complete

Step: `TW-19`

GOAL

A real signed-in user can actually reach the real Sprints page from Project Detail.

TECHNICAL CONTEXT

Pure wiring -- the real last piece connecting TW-01..18 into a real, reachable product flow.

FILE

`web/src/app/(app)/workspaces/[id]/projects/[projectId]/page.tsx`

EXACT LOCATION

In `web/src/app/(app)/workspaces/[id]/projects/[projectId]/page.tsx`, inside the real link row, right after the real "Board" link LC-33 added.

BEFORE

TSX

```
          Board
        </Link>
        <Link
          href={`/workspaces/${id}/projects/${project.id}/issues/new`}
```

IMPLEMENTATION

TSX

```
          Board
        </Link>
        <Link
          href={`/workspaces/${id}/projects/${project.id}/sprints`}
          className="text-blue-600 hover:underline dark:text-blue-400"
        >
          Sprints
        </Link>
        <Link
          href={`/workspaces/${id}/projects/${project.id}/issues/new`}
```

AFTER

TSX

```
          Board
        </Link>
```

```
        <Link
          href={`/workspaces/${id}/projects/${project.id}/sprints`}
          className="text-blue-600 hover:underline dark:text-blue-400"
        >
          Sprints
        </Link>
        <Link
          href={`/workspaces/${id}/projects/${project.id}/issues/new`}
```

WHY THIS WORKS

Placed between "Board" and "+ New issue" -- the real link row now reads left-to-right in the real order a student actually works: view the board, manage sprints, create an issue.

COMMANDS

BASH

```
npx tsc --noEmit && npm run build
```

VERIFY

BASH

```
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1 | grep -o
"Sprints"
```

EXPECTED RESULT

`tsc` / `build` both exit clean; the real link appears and navigates to the real Sprints page. TEAMWORK is complete: a real create-sprint form, a real sprint list with a real points summary, and real, forward-only Start/Complete lifecycle controls.